

Caso B LPR — Lectura de placas vehiculares a escala

Sherlyn Andrea Guzman Graciano¹

¹ Análisis de arquitectura para sistema de lectura de placas vehiculares

Abstract—

Es objetivo del siguiente documento es presentar un diseño conceptual de arquitectura cloud para un sistema de Lectura de Placas Vehiculares (LPR) a escala, implementado sobre Microsoft Azure. Se analizan los principales retos técnicos del caso de uso, se propone un flujo de procesamiento masivo orientado a eventos, se justifica la elección de cada servicio cloud frente a alternativas como Oracle Cloud Infrastructure (OCI), y se incluye una tabla de equivalencias para una implementación on-premises con herramientas open source.

Keywords— Sistemas, LPR, reconocimiento de imagenes, arquitectura cloud, Microsoft Azure, procesamiento, escalabilidad, IoT, visión artificial, alertas en tiempo real, on-premises.

I. INTRODUCCIÓN CASO DE USO

Los sistemas de reconocimiento de matrículas (LPR) basados en la nube se han convertido en herramientas esenciales para la gestión del tráfico y la seguridad. Determinar el mejor enfoque sigue siendo fundamental en el campo de la visión artificial.[1]

En comparación con los sistemas LPR tradicionales, el ALPR basado en la nube (cloud-based automatic license plate recognition) ofrece diversas ventajas que ayudan a maximizar la seguridad. Mediante cámaras u otros equipos especializados, el ALPR recopila imágenes de matrículas, que posteriormente se procesan y analizan en la nube utilizando algoritmos avanzados. La rápida expansión de las áreas urbanas y el consiguiente aumento del número de vehículos han intensificado la necesidad de sistemas eficientes de gestión del tráfico y seguridad. La tecnología de reconocimiento de matrículas (LPR) desempeña un papel crucial para abordar estos desafíos, ya que automatiza la identificación de vehículos mediante sus matrículas.

II. CONSIDERACIONES TÉCNICAS

Picos de trafico y escalabilidad.

Es importante tener en cuenta que el sistema debe procesar un flujo masivo de imágenes desde cámaras de tráfico, con

picos de carga que pueden multiplicar la demanda en horarios específicos. Este incremento genera un problema que se traduce en la necesidad de escalar recursos computacionales, la saturación del ancho de banda al transmitir miles de imágenes de alta resolución simultáneamente, y la sobrecarga de componentes internos las bases de datos durante una sobrecarga de peticiones.

Es por esto que al construir el sistema debe tenerse en cuenta: auto escalado basado en métricas como la profundidad de la cola de mensajes o el número de peticiones por segundo, permitiendo añadir o eliminar recursos de cómputo según la demanda para mantener tiempos de respuesta aceptables junto a políticas de re intento con retroceso exponencial junto con una cola de mensajes no entregables para garantizar que ningún evento se pierda incluso bajo estrés extremo.[2]

Precisión del reconocimiento y falsos positivos

Usualmente este tipo de sistemas pueden enfrentar variaciones significativas debido a la condición en que fueron recolectados los datos y si no están correctamente estandarizados lo que puede reducir la tasa de reconocimiento. Adicionalmente, los sistemas basados en OCR tradicional es posible que confundan caracteres similares como "O" y "0" o "B" y "8", y tienen dificultades con placas no estándar o de dos líneas. Un falso positivo en este contexto puede generar una alerta incorrecta sobre un vehículo legítimo, con consecuencias operativas o legales, por lo que se requiere un umbral de confianza configurable y un mecanismo de reentrenamiento continuo del modelo.

III. ARQUITECTURA

A. Diagramas de arquitectura

Los esquemas generados fueron los siguientes.

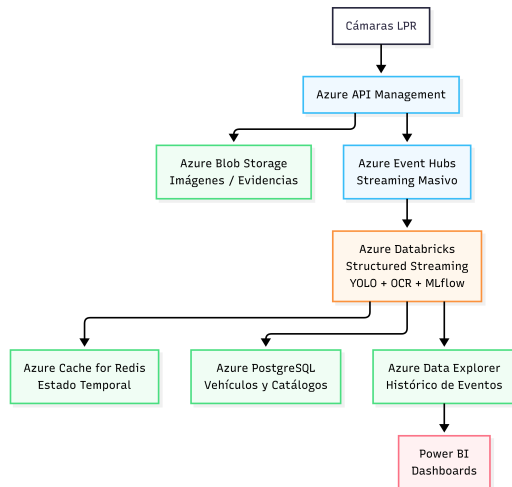


Fig. 1: Diagrama general con servicios de Azure

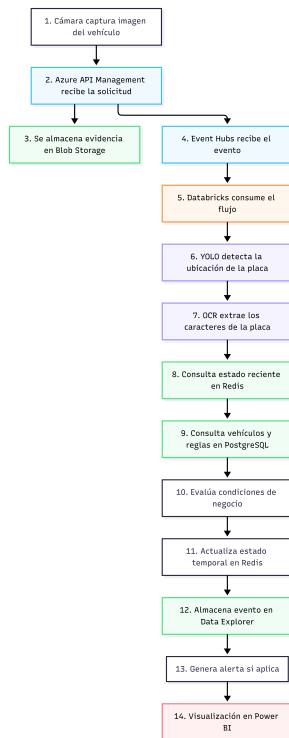


Fig. 2: Flujo de procesamiento y lógica de negocio

Se escogió Azure como proveedor de nube principalmente por las necesidades específicas del caso de uso, que gira en torno al procesamiento de imágenes con modelos de visión artificial y la gestión del ciclo de vida de esos modelos. La razón principal es que Azure Databricks ofrece una integración nativa con MLflow, esto es especialmente

relevante para un sistema LPR, donde los modelos YOLO y OCR deben ser monitoreados y mejorados continuamente a medida que se identifican errores de reconocimiento en producción.[3]

A diferencia de OCI, cuya fortaleza principal está en el rendimiento de bases de datos Oracle y cargas transaccionales, considero que Azure cuenta con un ecosistema más maduro para flujos de trabajo de inteligencia artificial.

Explicación del flujo de datos

El proceso inicia cuando una cámara de tráfico captura una imagen de un vehículo y la envía al sistema a través de Azure API Management, este actúa como punto de entrada seguro para la recepción de solicitudes. Posteriormente la imagen original es almacenada en Azure Blob Storage con el fin de conservar evidencia para llevar una trazabilidad de los datos, esto podría servir ya sea en auditorías, reprocesamiento futuro, entrenamiento de otros modelos de inteligencia artificial etc.

Mientras se almacena la imagen, simultáneamente se genera un evento que es enviado a Azure Event Hubs, este vendría siendo un servicio encargado de gestionar la ingesta de grandes volúmenes de datos en tiempo real.

Luego se pasa a una parte muy importante del flujo el cual se basa en el uso de Azure Databricks, este consume continuamente los eventos provenientes de Event Hubs mediante Apache Spark Structured Streaming. Una vez recibida la imagen, se ejecuta un modelo de visión artificial basado en YOLO, para localizar la placa del vehículo dentro de la imagen. Posteriormente, se aplica OCR para extraer el número de placa detectado. Se tomó la decisión de usar un modelo de YOLO ya que según el estado del arte de casos de usos en aplicaciones de LPR, en la mayoría de casos ha dado muy buenos resultados en términos de velocidad y precisión para este tipo de tareas.[1][4]

Ya teniendo la placa identificada, Databricks consulta Azure PostgreSQL para obtener información relacionada con el vehículo, incluyendo catálogos, listas de interés y reglas de negocio definidas por la organización. Adicionalmente, se consulta Azure Cache for Redis para acceder rápidamente al estado reciente de detecciones previas y evitar procesamientos o alertas duplicadas.

A continuación, se evalúan las reglas de negocio y condiciones de anomalía, como la identificación de vehículos reportados, placas incluidas en listas de seguimiento o comportamientos considerados sospechosos. Los resultados obtenidos son almacenados en Azure Data Explorer ya que este es de utilidad al momento de hacer consultas analíticas

sobre grandes volúmenes de eventos y series temporales.

Finalmente, la información procesada puede ser consumida por alguna interfaz de usuario si es necesario, como por ejemplo un dashboard de Power BI, y ser usada para la construcción de paneles de control y reportes operativos en tiempo real, permitiendo a los usuarios visualizar estadísticas de tráfico, detecciones realizadas, alertas generadas y tendencias históricas del sistema.

servicios cloud elegidos

Azure API Management Se seleccionó este servicio como punto de inicial del sistema debido a que permite centralizar la exposición de APIs, aplicar políticas de seguridad, autenticación, control de acceso y monitoreo del tráfico. Esto facilita la gestión de las solicitudes provenientes de múltiples cámaras y simplifica la administración de la plataforma.

Azure Blob Storage Se eligió Azure Blob Storage para almacenar las imágenes capturadas por las cámaras antes y después de ser procesadas. Según la documentación de Microsoft, Blob Storage está optimizado para almacenar grandes cantidades de datos no estructurados, como imágenes, videos y documentos

Azure Event Hubs En este caso, este servicio funciona como una capa de ingesta de eventos de la solución. Todas las imágenes capturadas por las cámaras generan eventos que son enviados a Event Hubs, donde quedan disponibles para ser consumidos posteriormente por el servicio de Azure Databricks.

Azure Databricks Es una plataforma unificada y basada en la nube para el análisis de macrodatos (Big Data) y la inteligencia artificial (IA), desarrollada conjuntamente por Microsoft y Databricks. Una de las principales ventajas es la integración nativa con MLflow, una herramienta que facilita el seguimiento de experimentos, el versionado de modelos, el registro de modelos y su despliegue a producción. Esto fue el motivo de su elección ya que resulta especialmente útil para el sistema propuesto, ya que los modelos en general deben ser monitoreados y pueden ser mejorados continuamente etc.

Azure Databricks se seleccionó para almacenar la información estructurada del sistema, como lo pueden ser los registros de vehículos, las listas de seguimiento, datos y reglas de negocio establecidas en el flujo. PostgreSQL es una base de datos relacional ampliamente utilizada que

permite realizar consultas complejas y mantener la integridad de los datos mediante relaciones entre tablas. En este caso se elige una DB relacional por temas de escalabilidad de esta estructura de datos en específico ya que se basa en relaciones específicas y estrictas, además al utilizar la versión administrada de Azure, se simplifica la operación de la plataforma.[3]

Azure Cache for Redis Al ser un servicio en la nube totalmente administrado por Microsoft que proporciona una caché de datos en memoria se eligió por su capacidad para almacenar información temporal de las detecciones recientes y acelerar consultas frecuentes dentro del sistema.

Azure Data Explorer Este es un servicio de análisis de datos totalmente administrado para analizar grandes volúmenes de streaming de datos procedentes de aplicaciones, sitios web, dispositivos IoT, etc. en tiempo real. Permite mediante la exploración de los datos identificar patrones, anomalías y tendencias en los datos con rapidez, y ese es el motivo de su elección.

Consideraciones de escalabilidad, disponibilidad y seguridad del diseño propuesto.

Escalabilidad Es importante que número de particiones en Event Hubs se dimensione desde el inicio considerando el volumen máximo aproximado esperado de cámaras, ya que aumentarlas después implica reconfiguración del pipeline. Además de esto clúster de Databricks debe configurarse con auto-scaling habilitado y un mínimo de workers activos para evitar cold starts que introduzcan latencia en el streaming. Es importante también que Redis deba monitorearse con alertas sobre uso de memoria para evitar la generación de alertas duplicadas o inconsistencias en el estado temporal. Por último, con respecto a la DB, PostgreSQL debe contar con índices adecuados sobre los campos de búsqueda más frecuentes (número de placa, listas de seguimiento) para mantener tiempos de consulta aceptables bajo alta concurrencia.

Disponibilidad Como se mencionó anteriormente se escogió el servicio de qEvent Hubs actúa como intermediario entre las cámaras y el procesamiento, es por esto que un fallo temporal en Databricks no implicaría pérdida de imágenes, ya que los eventos quedan almacenados esperando ser procesados. Pero para aprovechar esto, sería importante configurar un tiempo de retención suficientemente largo como para cubrir el tiempo que tomaría recuperar Databricks ante un fallo. Adicionalmente, se recomendaría

también habilitar una cola de mensajes no entregables, que capture automáticamente aquellos eventos que fallaron repetidamente en su procesamiento, evitando que se pierdan de forma silenciosa sin dejar rastro

Finalmente seria importante definir cuánto tiempo puede tolerar el sistema estar caído y cuántos datos puede permitirse perder ante un fallo (por ejemplo, máximo 5 minutos de inactividad y pérdida de no más de 1 minuto de eventos). Esto es especialmente crítico en PostgreSQL para evitar perdida de información o tracking de las alertas.

Seguridad Como el sistema maneja imágenes de vehículos y datos vinculados a personas, la seguridad debe aplicarse. Las cámaras o clientes externos que consuman la API deben autenticarse con credenciales rotativas ante API Management, evitando que cualquier fuente no autorizada pueda inyectar eventos al pipeline. El acceso a Blob Storage debe limitarse a la red privada (VNet), eliminando exposición pública de las imágenes capturadas. Los datos en PostgreSQL deben cifrarse en reposo y todo acceso debe quedar registrado en logs de auditoría, considerando que contiene listas de seguimiento y reglas de negocio sensibles. Los secretos, claves de conexión y tokens de acceso deben gestionarse mediante Azure Key Vault en lugar de estar embebidos en el código, y toda comunicación entre servicios debe realizarse sobre canales cifrados (HTTPS).

REFERENCES

- 1. Castillo-Cara Manuel, others . Cloud-Based License Plate Recognition: A Comparative Approach Using YOLO Versions 5, 7, 8, and 9 Object Detection *Information*. 2025;16:57.
- 2. Ilimit . ¿Cómo funciona el Autoscaling? 2023.
- 3. Microsoft . Azure Databricks
- 4. Infotecnico . LPR: Vigilancia automatizada de vehículos

IV. ENTORNO ON-PREMISES

Servicio Azure	Alternativa On-Premises	Justificación
Azure Event Hubs	Apache Kafka	Kafka es la herramienta open source por mas usada para el manejo de colas, además de que si el consumidor falla los mensajes no se pierden ya que Kafka los retiene hasta que puedan ser procesados.
Azure Databricks (Spark + MLflow)	Apache Spark + MLflow	Se escoge esta combinación porque Spark ya viene integrado en Databricks, por lo que la lógica de procesamiento es directamente portable, y MLflow permite seguir gestionando el ciclo de vida de los modelos YOLO y OCR de forma local sin cambiar la forma de trabajo.
Azure API Management	NGINX	Permite centralizar la entrada de solicitudes al sistema, gestionar autenticación y distribuir carga sin requerir infraestructura compleja ni licencias.
Azure PostgreSQL	PostgreSQL	es el mismo motor, por lo que no hay cambios en el esquema ni en las consultas, solo se pasa a administrarlo directamente en el servidor local.
Azure Cache for Redis	Redis	Se escoge Redis porque Azure Cache for Redis es en realidad Redis administrado, la funcionalidad es la misma y simplemente se instala directamente en el servidor propio.
Azure Data Explorer	Apache Flink	Flink porque permite procesar y consultar grandes volúmenes de eventos en tiempo real sobre infraestructura propia, cubriendo la necesidad de analizar el flujo continuo de detecciones y alarmas.
Azure Blob Storage	MinIO	Permite exponer una API compatible con S3, lo que significa que el código que ya interactúa con Blob Storage puede adaptarse fácilmente sin reescribir la lógica de almacenamiento.

Table 1: Equivalencias entre servicios Azure y alternativas on-premises open source